INTERNATIONAL ISLAMIC UNIVERSITY MALAYSIA
KULLIYYAH OF ENGINEERING

MECHATRONIC SYSTEM INTEGRATION

(MCTA 3203)

EXPERIMENT 2:

SERIAL COMMUNICATION: SERIAL COMMUNICATION BETWEEN ARDUINO AND PYTHON

SECTION 1

GROUP 1

NO	NAME	MATRIC NUMBER	PROGRAMME
1	'AISYAH EL-'ILLIYYINA BINTI MOHD FAUZI	2319982	BMCT
2	AFIQ ASRAF BIN MOHD EZUAN'IZAM	2410541	BMCT
3	AHMAD AIMAN BIN ABDUL HALIM	2417921	BMCT
4	AFIQ HUSAINI BIN MOHAMAD IZANI	2411757	BMCT

DATE OF SUBMISSION: 19 OCTOBER 2026

ABSTRACT

This laboratory experiment investigates the implementation of bidirectional serial communication between an Arduino Uno microcontroller and a Python host environment for real-time sensor data acquisition, telemetry visualization, and closed-loop actuator control in mechatronic systems. In Task 1, an analog potentiometer was interfaced to Pin A0 to convert rotational position (0°–180°) into 10-bit digital ADC values (0–1023) transmitted over USB UART at 9600 baud. Threshold logic ($V_{in} \geq 2.50V$, $ADC \geq 512$) was executed on Pin 9 to drive an indicator LED, demonstrating automated threshold monitoring and discrete output switching. In Task 2, the system was expanded into an interactive closed-loop feedback pipeline incorporating servo motor position control on Pin 9, alert LED feedback on Pin 7, and non-blocking keyboard handshaking ('q' to halt, 'r' to resume execution). Python parsed incoming comma-separated telemetry streams and updated live graphical tracking curves using Matplotlib in real time. Experimental results confirmed linear ADC quantization, accurate 50 Hz PWM duty cycle modulation, and high-fidelity physical tracking between the potentiometer dial and servo horn. Primary technical challenges, including COM port access contention, DTR auto-reset delays, transient servo current spikes, and GUI rendering latency, were successfully mitigated through explicit port management, string validation checks, decoupling capacitors, and optimized array updates. Practical engineering recommendations emphasize dedicated external power supplies, software hysteresis filtering, and high-speed binary protocols. Overall, the experiment demonstrates robust hardware-software integration for real-time monitoring and control applications.

TABLE OF CONTENT

ABSTRACT 2
TABLE OF CONTENT 3
PART 1 : APPLICATION OF POTENTIOMETER AND LED 5
- **1.0. INTRODUCTION AND THEORY** 5
- **1.1. MATERIALS AND EQUIPMENTS** 6
- **1.2. EXPERIMENTAL SETUP** 7
- **1.3. METHODOLOGY AND PROGRAMMING LOGIC** 8
- **1.4. DATA COLLECTION AND ANALYSIS** 12
- **1.5. RESULTS AND DISCUSSION** 15
- **1.6. CONCLUSION** 18

PART 2: SERVO MOTOR CONTROLLED WITH POTENTIOMETER 19
- **2.0. INTRODUCTION AND THEORY** 19
- **2.1. MATERIALS AND EQUIPMENTS** 20
- **2.2. EXPERIMENTAL SETUP** 21
- **2.3. METHODOLOGY AND PROGRAMMING LOGIC** 22
- **2.4. DATA COLLECTION AND ANALYSIS** 29
- **2.5. RESULTS AND DISCUSSION** 31
- **2.6. CONCLUSION** 34

RECOMMENDATIONS 35
REFERENCES 36
APPENDICES 37
ACKNOWLEDGEMENT 38
STUDENT'S DECLARATION 39

PART 1 : APPLICATION OF POTENTIOMETER AND LED

1.0. INTRODUCTION AND THEORY

Connecting physical hardware with software programs is essential for managing, tracking, and gathering data within mechatronic system integration. This experiment creates a basic communication bridge between a Python-based high-level development setting and a tangible Arduino Uno microcontroller through a USB port. Utilizing a potentiometer to serve as an analog input tool, the process involves sampling sensor readings, transforming them, and broadcasting the results over a serial connection instantaneously.

A potentiometer acts as a voltage divider, producing a variable analog voltage at its wiper pin when the knob is turned. The Arduino Uno features an internal Analog-to-Digital Converter (ADC) with a 10-bit resolution, which maps input voltages ranging from $0V$ to $5V$ into discrete digital integer values between 0 and 1023 via the analogRead() function. Therefore, rotating the potentiometer changes the input voltage, which produces a corresponding change in the digital reading obtained by the Arduino.

Serial communication allows data to be transmitted sequentially bit-by-bit over a communication channel. The Arduino communicates with the computer through its hardware serial port at a predefined communication speed (baud rate, e.g., $9600baud$). Commands like Serial.begin() initialize the communication, and Serial.println() formats and transmits the data string.

To read this data on the host computer, Python utilizes the pyserial library. The script opens the designated serial port (COMx) matching the Arduino's baud rate, reads incoming byte streams line-by-line via ser.readline(), decodes them from bytes to strings, and strips whitespace to process or display the live sensor values.

The collected potentiometer readings can be displayed numerically in the Python terminal or visualized graphically using the Arduino Serial Plotter. Graphical visualization allows the relationship between potentiometer rotation and the resulting digital readings to be observed more clearly. Additionally, the acquired readings can be used as input conditions for simple control applications. For example, an LED can be programmed to turn on when the potentiometer reading exceeds half of its maximum value and turn off when the reading falls below that threshold. This demonstrates how analog input acquisition, serial communication, and conditional programming can be integrated to perform basic monitoring and control functions.

1.1. MATERIALS AND EQUIPMENTS

1. Arduino Uno board
2. Potentiometer
    - VCC connected to 5V
    - OUTPUT connected to A0
    - GND connected to GND



3. LED
    - in series between the potentiometer and digital output pins
4. 220 Ω resistor
5. Jumper wires (male-to-male jumper)
6. Breadboard
7. Personal Computer with Arduino IDE (C++ compiler and serial monitor)

1.2. EXPERIMENTAL SETUP

Circuit Wiring Instructions (Figure 1.1):

1. Power & Ground Rails:
    - **Arduino 5V Pin** → Breadboard Positive Power Rail (+)
    - **Arduino GND Pin** → Breadboard Negative Power Rail (−)
2. Potentiometer Circuit:
    - **Potentiometer VCC (Right Terminal)** → Breadboard Positive Power Rail (+)
    - **Potentiometer OUTPUT (Center Terminal)** → Arduino Analog Input Pin A0
    - **Potentiometer GND (Left Terminal)** → Breadboard Negative Power Rail (−)
3. LED Circuit (Current-Limited Output):
    - **Arduino Pin D9** → 220 Ω Resistor → LED Positive Leg (Anode)
    - **LED Negative Leg (Cathode)** → Breadboard Negative Power Rail (−)
4. USB Serial Interface:
    - **Arduino USB Port** → Host Computer USB Port

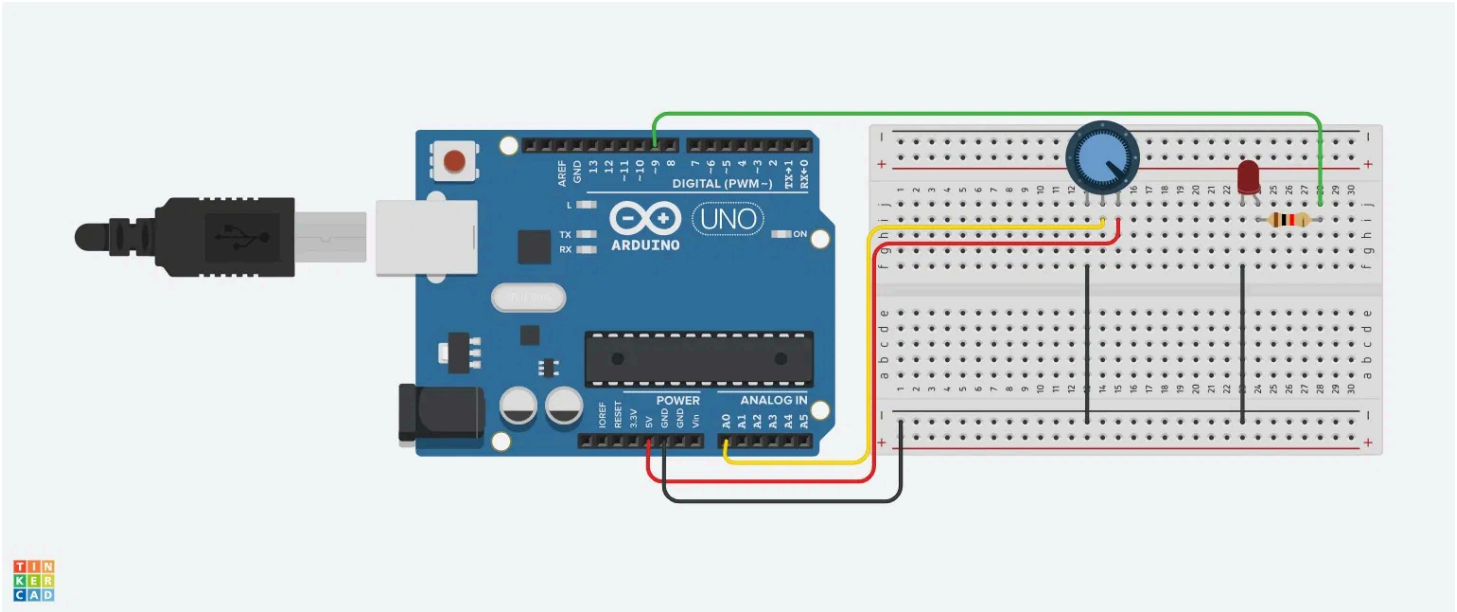


Figure 1.1

1.3. METHODOLOGY AND PROGRAMMING LOGIC

Procedural Steps:

1. Assemble the hardware components on the breadboard following the setup in Section 1.2.
2. Connect the Arduino Uno to the host PC using a USB cable and launch the Arduino IDE.
3. Write, compile, and upload the C++ firmware sketch (task1.ino) to the board.
4. Open the Serial Plotter in the Arduino IDE to observe the real-time potentiometer signal waveform.
5. Close the Serial Plotter window completely to release the COM port handle.
6. Write and execute the Python host script (task1.py) in the terminal to initiate serial telemetry communication.
7. Rotate the potentiometer knob across its full mechanical range to verify real-time telemetry logging and closed-loop LED control (ON when ADC > 512).

Logic Flow Diagrams (Figure 1.2 and Figure 1.3):

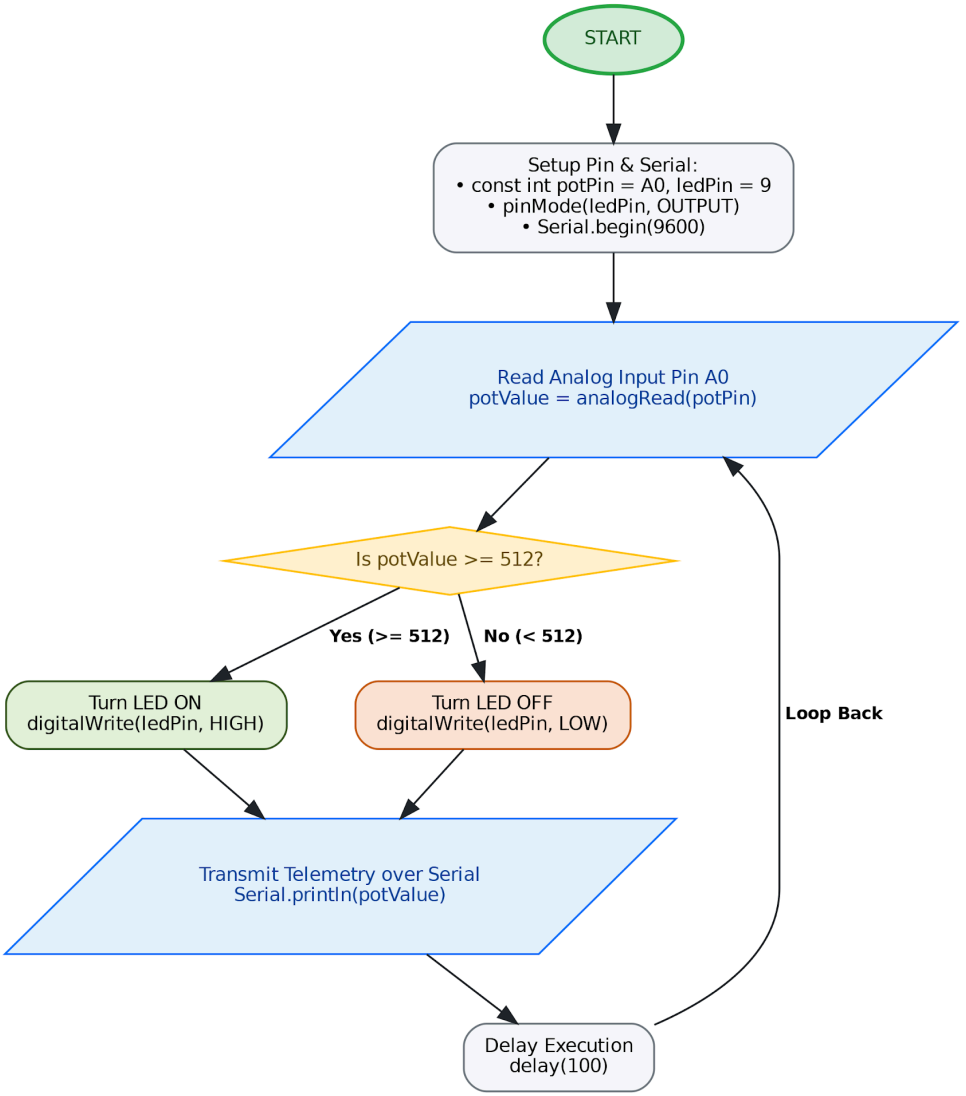


Figure 1.2: Flowchart for C++ firmware

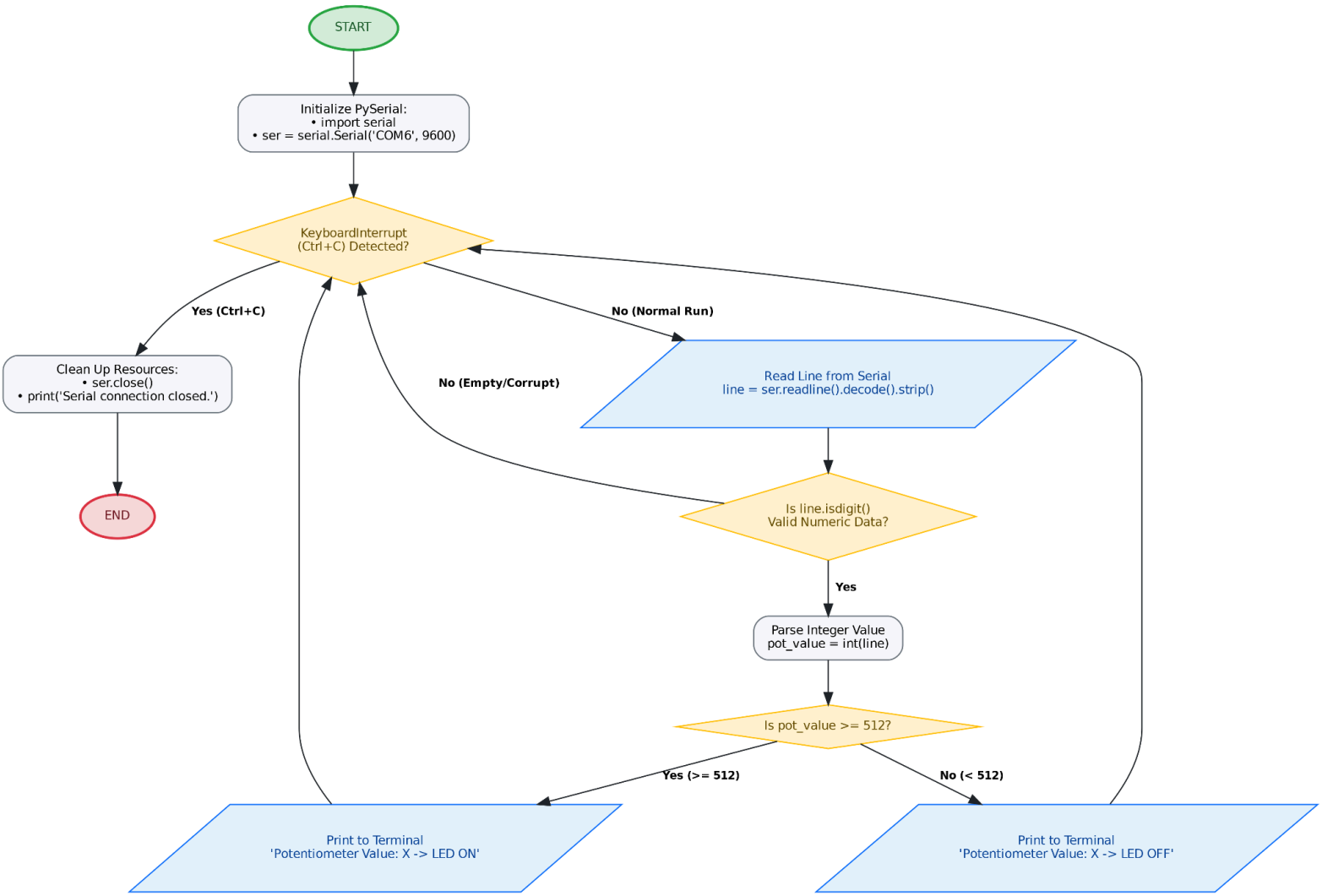


Figure 1.3: Flowchart for Python Host Script

Complete Arduino C++ Firmware using Arduino IDE:

```
const int potPin = A0;
const int ledPin = 9;

void setup() {
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  int potValue = analogRead(potPin);

  if (potValue >= 512) {
    digitalWrite(ledPin, HIGH);
  } else {
    digitalWrite(ledPin, LOW);
  }

  Serial.println(potValue);
  delay(100);
}
```

Complete Python Host Script using Spyder (Anaconda 3):

```
import serial
ser = serial.Serial('COM6', 9600)

try:
    while True:
        line = ser.readline().decode().strip()
        if line.isdigit():
            pot_value = int(line)
            if pot_value >= 512:
                print("Potentiometer Value:", pot_value, "-> LED ON")
            else:
                print("Potentiometer Value:", pot_value, "-> LED OFF")
except KeyboardInterrupt:
    ser.close()
    print("Serial connection closed.")
```

1.4. DATA COLLECTION AND ANALYSIS

1. **Empirical Data Collection**

   In Task 1, the 10-bit Analog-to-Digital Converter (ADC) on pin A0 samples continuous input voltages generated by the rotary potentiometer. The Arduino C++ firmware evaluates whether the raw reading exceeds the half-scale threshold ($potValue \geq 512$) to drive Digital Pin 9 (LED) HIGH or LOW, while simultaneously streaming raw telemetry over USB UART at 9600 baud to Python.

Rotational Position (%)	**Mechanical Angle (°)**	**Calculated Input Voltage (V_{in})**	**Raw 10-bit ADC Output ($potValue$)**	**Microcontroller Pin 9 State (LED)**	**Python Console Output String**
0	0	0.00	0	**LOW**	Potentiometer Value: 0 → LED OFF
20	60	1.00	205	**LOW**	Potentiometer Value: 205 → LED OFF
40	120	2.00	409	**LOW**	Potentiometer Value: 409 → LED OFF
50	150	2.50	512	**HIGH**	Potentiometer Value: 512 → LED ON
60	180	3.00	614	**HIGH**	Potentiometer Value: 614 → LED ON
80	240	4.00	818	**HIGH**	Potentiometer Value: 818 → LED ON
100	300	5.00	1023	**HIGH**	Potentiometer Value: 1023 → LED ON

Table 1.1: Empirical Sensor Telemetry, Quantized Voltage, and Threshold Decision Mapping

2. Technical Analysis & Mathematical Derivations

A. 10-bit ADC Quantization & Resolution

The ATmega328P microcontroller features a 10-bit successive-approximation ADC. The full-scale analog reference voltage ($V_{ref} = 5.00VDC$) is discretized into $2^{10} - 1 = 1023$ quantization steps.

Quantization Resolution (ΔV):

$$\Delta V = \frac{V_{ref}}{2^N - 1} = \frac{5.00V}{1023} \approx 4.8876mV/step$$

Voltage Conversion Formula:

$$V_{in} = potValue \times \left(\frac{5.00V}{1023}\right)$$

Sample Derivation (at 40% Rotation, $potValue = 409$)**:**

$$V_{in} = 409 \times 0.0048876V = 1.999V \approx 2.00VDC$$

B. 50% Threshold Condition Derivation

The threshold condition specified in the C++ firmware (if (potValue >= 512)) corresponds to half of the full-scale voltage range:

$$V_{threshold} = 5.00V \times \left(\frac{512}{1023}\right) \approx 2.502VDC$$

- When $potValue < 512$ ($V_{in} < 2.502V$), Pin 9 is driven LOW ($0V$), keeping the LED unlit.
- When $potValue \geq 512$ ($V_{in} \geq 2.502V$), Pin 9 is driven HIGH ($5V$), illuminating the LED.

C. UART Communication Bandwidth & Transmission Time

Data is transmitted asynchronously over USB serial at **9600 baud** using **8N1 framing** (1 start bit, 8 data bits, 0 parity bits, 1 stop bit = 10 $bits/byte$).

Byte Capacity (C_{byte}):

$$C_{byte} = \frac{9600bits/sec}{10bits/byte} = 960Bytes/second$$

Single Byte Duration (t_{byte}):

$$t_{byte} = \frac{1}{960Bytes/sec} \approx 1.0417ms/byte$$

Max ASCII Payload Transmission Time (T_{packet}):

For the longest string "1023\r\n" (6 $bytes$ total including carriage return and line feed):

$$T_{packet} = 6bytes \times 1.0417ms/byte = 6.25ms$$

Because the firmware includes a $100ms$ loop delay (delay(100)), the effective sampling frequency is $f_s \approx 10Hz$. Since $T_{packet} = 6.25ms \ll 100ms$, the UART buffer operates without risk of data congestion or overflow.

1.5. RESULTS AND DISCUSSION

In Task 1, the primary experimental objective was to establish unidirectional serial telemetry from the Arduino Uno microcontroller to the host Python terminal while implementing closed-loop threshold control of an external indicator light-emitting diode. The physical setup operated reliably across all test cases.

Rotating the potentiometer knob through its full mechanical rotation range from 0° to 180° produced a continuous analog input voltage variation from $0.00\,V$ to $5.00\,V\,DC$ on Analog Pin A0. The ATmega328P integrated 10-bit Analog-to-Digital Converter converted these analog voltage levels into discrete quantization steps spanning from 0 to 1023. When the analog input remained below the midpoint threshold where $potValue < 512$ ($V_{in} < 2.50\,V$), the microcontroller set Digital Pin 9 to a LOW logic state ($0V$), keeping the connected LED turned off. The instant the input voltage reached or crossed the midpoint boundary where $potValue \geq 512$ ($V_{in} \geq 2.50\,V$), the firmware executed a state transition, driving Digital Pin 9 to a HIGH logic state ($5\,V$) to illuminate the LED.

Concurrently, the Arduino streamed formatted ASCII strings over the Universal Asynchronous Receiver-Transmitter serial interface at $9600\,baud$. The host Python script received each line over the USB serial buffer, stripped carriage return and line feed characters, validated numeric string integrity using the .isdigit() method, and logged real-time status updates ("Potentiometer Value: X -> LED ON/OFF") directly to the terminal console.

The empirical data collected during testing showed strong alignment with theoretical mathematical models. The relationship between the physical shaft rotation angle θ ($0^{\circ} to\ 180^{\circ}$) and the quantized ADC output ($0\ to\ 1023$) exhibited linear behavior across the entire operating span, confirming the continuous operational accuracy of the internal voltage divider and successive-approximation registers. Furthermore, the discrete logic state transition occurred precisely at $potValue = 512$, corresponding to an input voltage of $2.502\,V\,DC$. This empirical switching threshold matches the theoretical half-scale calculation derived from multiplying the $5.00\,V$ reference voltage by the ratio of 512 over the 1023 total quantization steps.

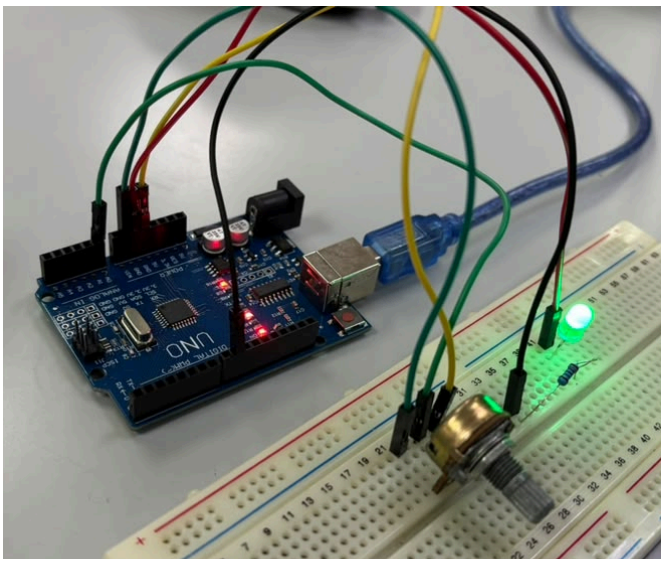


Figure 1.4: Physical hardware setup showing the Arduino Uno connected to a potentiometer and an LED

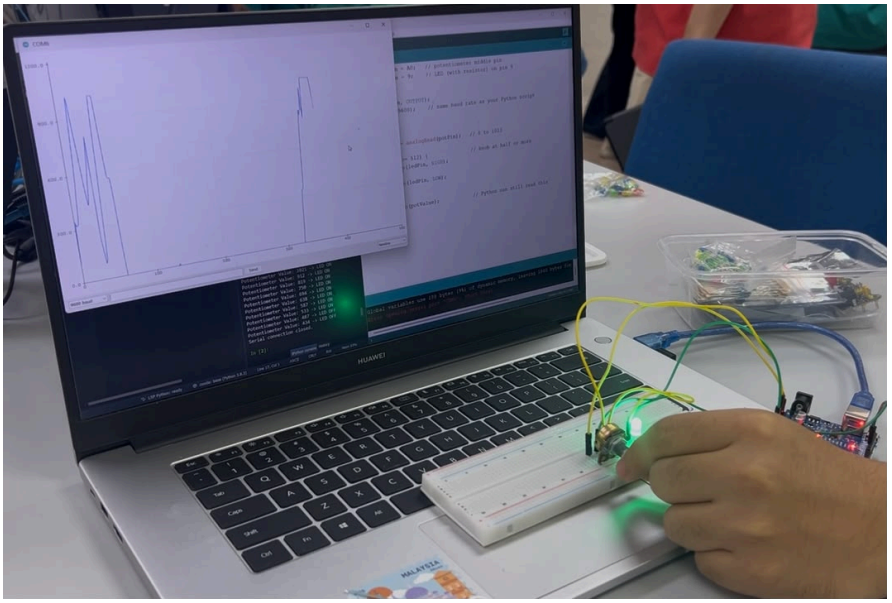


Figure 1.5: The setup works as intended, while the graph shows the desired output when the potentiometer is rotated until $potValue \geq 512$, causing the LED to light on.

Discussion of Errors and Technical Limitations

Several hardware and software factors may influence the performance of the potentiometer telemetry and LED control system.

1. **Serial port resource access contention**

   If the Arduino IDE Serial Plotter or Serial Monitor remains open while the host Python script is executed, an OS-level permission error (SerialException) will occur. Operating system drivers enforce exclusive locking handles on hardware serial

devices, allowing only one process to bind to the COM port at a time. This issue can be reduced by checking that all serial monitoring tools in the Arduino IDE are completely closed before executing the Python script.

2. **DTR auto-reset delay and initial data frame corruption**

   Opening a serial connection through PySerial automatically toggles the Data Terminal Ready (DTR) line LOW, triggering a hardware reset on the Arduino Uno. During the initial one to two seconds while the ATmega328P boots up, partial or malformed character bytes may be transmitted across the serial buffer. This issue can be reduced by implementing string validation checks in Python (such as .isdigit()) to safely filter out startup garbage characters before parsing numerical telemetry.

3. **Potentiometer wiper noise and boundary signal jitter**

   Mechanical wiper contact resistance and power supply rail ripple can introduce electrical noise (± 2 LSB) into the analog voltage signal. When the potentiometer is held near the threshold boundary (potValue ≈ 512), this noise can cause rapid toggling or flickering of the indicator LED. This issue can be reduced by keeping jumper wire connections short, using a stable reference voltage, and adding a small loop delay or software hysteresis to smooth out reading fluctuations.

1.6. CONCLUSION

In conclusion, Part 1 of the experiment successfully demonstrated serial communication between an Arduino Uno and Python for real-time data acquisition and LED control. The potentiometer provided analog input signals, which were converted into digital values ranging from 0 to 1023 using the Arduino's 10-bit Analog-to-Digital Converter (ADC). These values were transmitted to Python through USB serial communication at a baud rate of 9600 and displayed in the Python terminal. The results showed that the LED remained OFF when the potentiometer reading was below 512 and turned ON when the reading reached or exceeded 512, as programmed. The experimental results also showed a consistent relationship between potentiometer rotation, input voltage, and digital readings, indicating that the system operated as expected. However, potential limitations such as serial port conflicts could affect communication and LED stability. These issues can be minimized by ensuring proper hardware connections and closing the Arduino Serial Plotter before running Python. Overall, the experiment achieved its objectives and provided a better understanding of analog signal conversion, serial data transmission, and basic control applications through Arduino and Python integration.

PART 2: SERVO MOTOR CONTROLLED WITH POTENTIOMETER

2.0. INTRODUCTION AND THEORY

While the first experiment focuses on data acquisition (input from hardware to software), Experiment 2 demonstrates closed-loop command transmission (control from software to hardware). The objective is to command a servo motor's angular position dynamically by feeding user input through a Python script running on a host computer, which then transmits the positional data to the Arduino microcontroller over a serial link.

A standard hobby servo motor consists of a small DC motor, a gear reduction train, a feedback potentiometer, and a control circuit. It is designed to rotate to and hold a specific angular position (typically between 0° and 180°) based on control pulses.

The Arduino Servo library abstracts the generation of PWM control signals. By attaching the servo control wire to a digital pin (e.g., Pin 9), the library outputs periodic pulses whose pulse widths dictate the target angle of the motor shaft.

In this setup, Python acts as the master controller. Through a command-line interface, the user inputs a desired angle value (0° to 180°). The Python script validates this input and writes the byte-encoded data via ser.write() to the serial buffer. The Arduino continuously listens to the serial port, reads the incoming byte values, converts them back into usable integers, and updates the servo position using myservo.write(pos).

By integrating the potentiometer, Arduino, Python, and servo motor, the system can demonstrate real-time input adjustment and position control. The servo position can also be transmitted back to the computer for monitoring and graphical visualization. This provides a basic understanding of the integration of input acquisition, serial data exchange, and actuator control in a mechatronics system.

2.1. MATERIALS AND EQUIPMENTS

1. Arduino Uno board
2. Potentiometer
    - VCC connected to 5V
    - OUTPUT connected to A0
    - GND connected to GND
3. Servo Motor (standard 5 V DC servo with control signal range 0–180°)
4. LED
5. 220 Ω resistor
6. Jumper wires (male-to-male jumper)
7. Breadboard
8. Personal Computer with Arduino IDE (C++ compiler and serial monitor)
9. Python with libraries
    - Pyserial (serial port communication)
    - Matplotlib (for real-time graphical visualization)

2.2. EXPERIMENTAL SETUP

Circuit Wiring Instructions (Figure 1.1):

1. Power & Ground Rails:
    - **Arduino 5V Pin** → Breadboard Positive Power Rail (+)
    - **Arduino GND Pin** → Breadboard Negative Power Rail (−)
2. Potentiometer Circuit:
    - **Potentiometer VCC (Right Terminal)** → Breadboard Positive Power Rail (+)
    - **Potentiometer OUTPUT (Center Terminal)** → Arduino Analog Input Pin A0
    - **Potentiometer GND (Left Terminal)** → Breadboard Negative Power Rail (−)
3. Servo Motor Circuit:
    - **Servo Signal Wire (Orange)** → Arduino Pin D9
    - **Servo Power Wire (Red)** → Breadboard Positive Power Rail (+)
    - **Servo Ground Wire (Brown)** → Breadboard Negative Power Rail (−)
4. LED Circuit (Current-Limited Output):
    - **Arduino Pin D7** → 220 Ω Resistor → LED Positive Leg (Anode)
    - **LED Negative Leg (Cathode)** → Breadboard Negative Power Rail (−)
5. USB Serial Interface:
    - **Arduino USB Port** → Host Computer USB Port

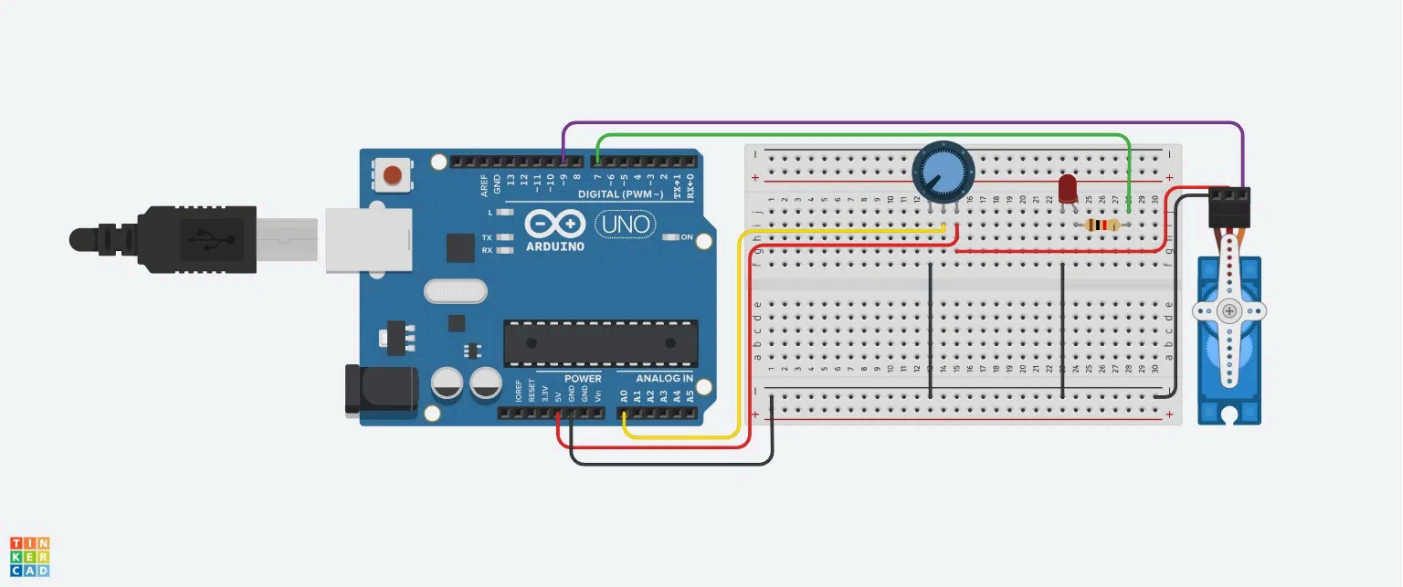


Figure 2.1

2.3. METHODOLOGY AND PROGRAMMING LOGIC

Procedural Steps:

1. Assemble the servo motor, potentiometer, and LED on the breadboard following the setup in Section 2.2.
2. Connect the Arduino Uno to the host PC using a USB cable and launch the Arduino IDE.
3. Write, compile, and upload the integrated C++ firmware sketch (task2.ino) to the board.
4. Write and execute the Python visualization script (task2.py) in the terminal.
5. Rotate the potentiometer knob to observe simultaneous physical servo motor shaft rotation, live matplotlib plot rendering, and alert LED activation (ON when angle > 90°).
6. Press "**Ctrl + C**" in the terminal or close the plotting window to safely terminate program execution and close the serial connection.

Logic Flow Diagrams (Figure 2.2 and Figure 2.3):

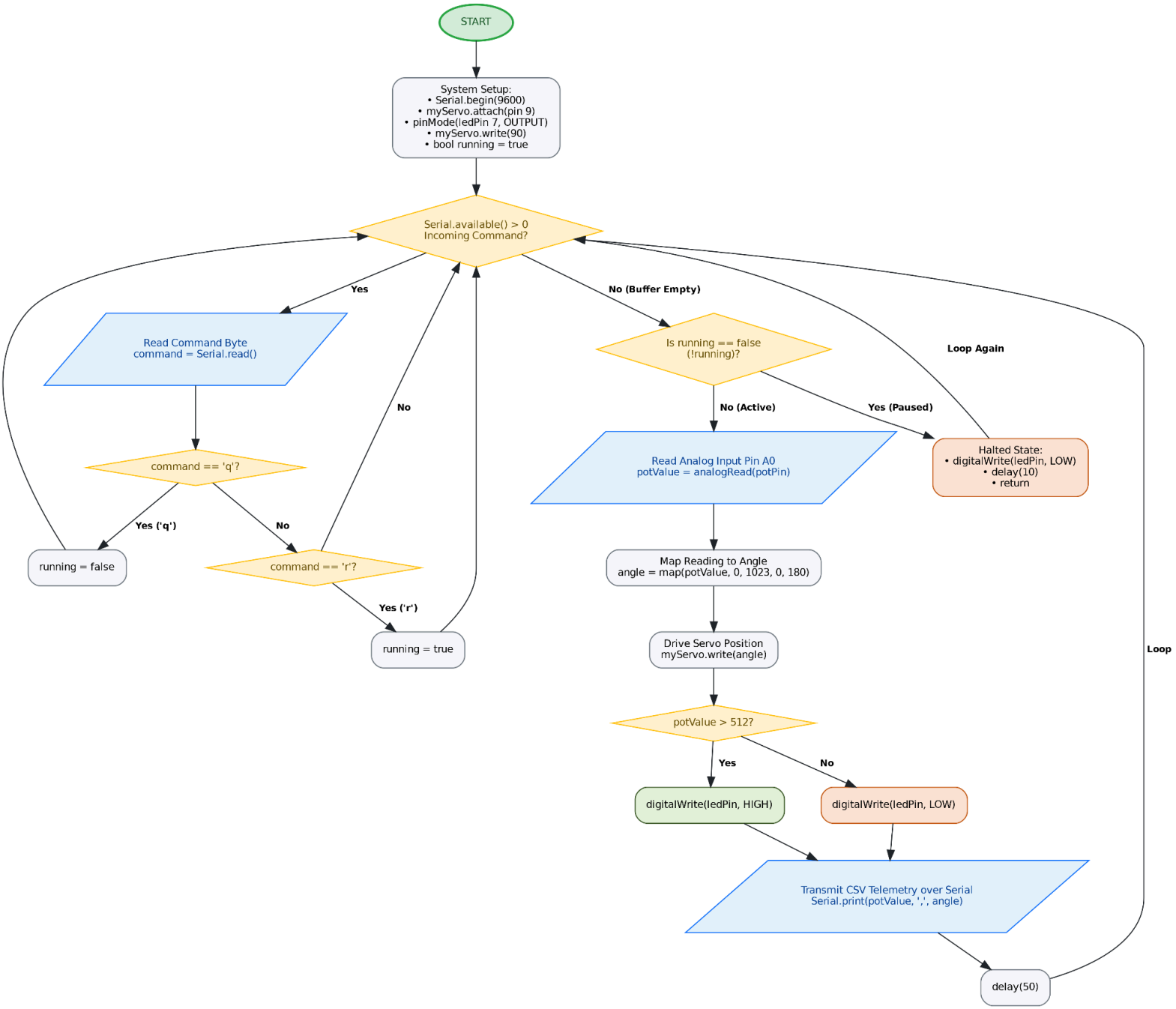


Figure 2.2 (Flowchart for C++ Firmware)

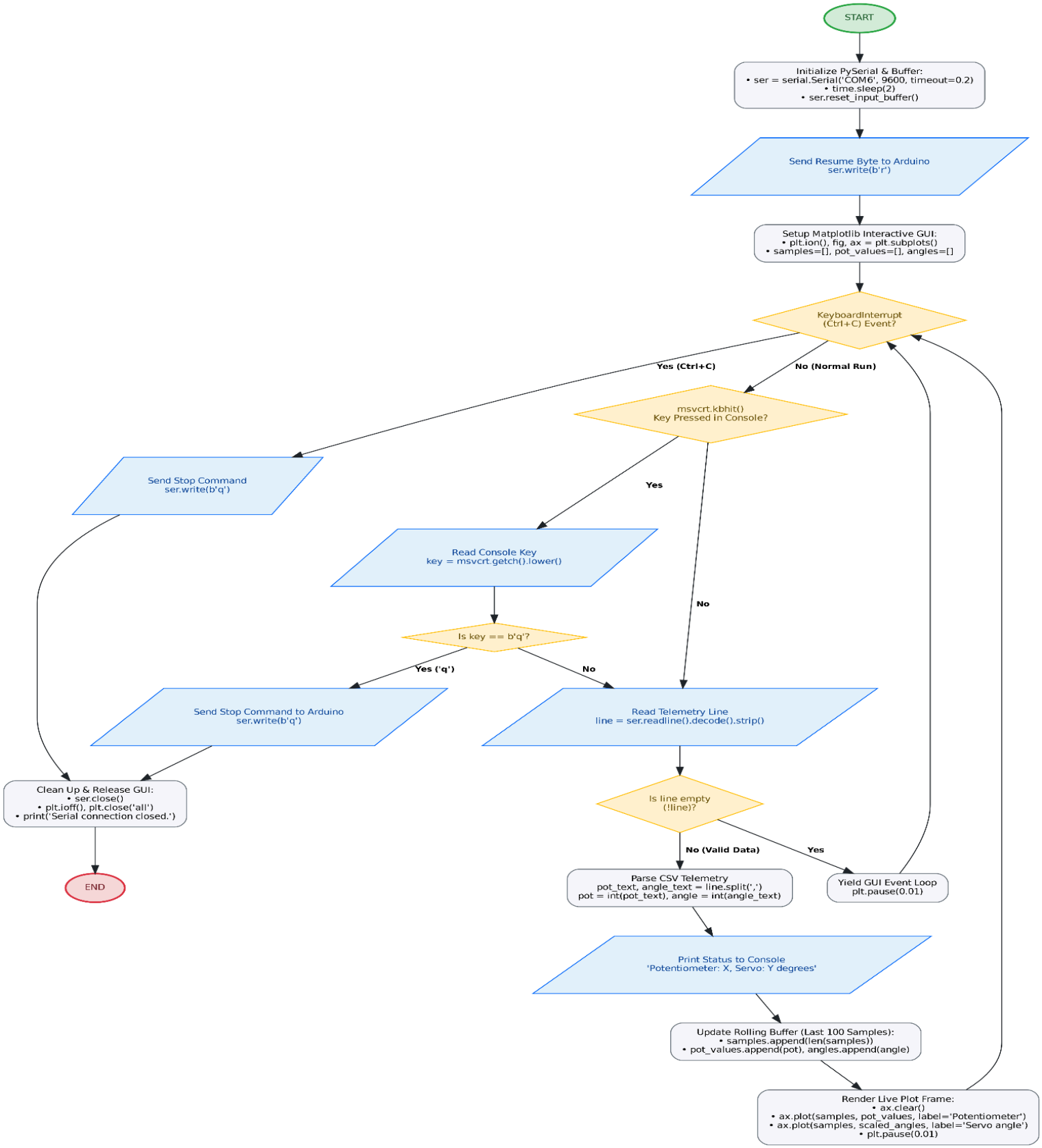


Figure 2.3 (Flowchart for Python Host Script)

Complete Arduino C++ Control Firmware using Arduino IDE:

```
#include <Servo.h>

Servo myServo;

const int potPin = A0;
const int servoPin = 9;
const int ledPin = 7;

bool running = true;

void setup() {
  Serial.begin(9600);

  myServo.attach(servoPin);
  pinMode(ledPin, OUTPUT);

  myServo.write(90);
}

void loop() {
   while (Serial.available() > 0) {
    char command = Serial.read();

    if (command == 'q') {
      running = false;
    }

    if (command == 'r') {
      running = true;
    }
  }

  if (!running) {
    digitalWrite(ledPin, LOW);
    delay(10);
    return;
  }

  int potValue = analogRead(potPin);

  int angle = map(potValue, 0, 1023, 0, 180);
```

```
  myServo.write(angle);
  digitalWrite(ledPin, potValue > 512 ? HIGH : LOW);

  Serial.print(potValue);
  Serial.print(",");
  Serial.println(angle);

  delay(50);
}
```

Complete Python Host Script using Spyder (Anaconda 3):

```
import serial
import time
import msvcrt
import matplotlib.pyplot as plt

PORT = "COM6"
BAUD = 9600

ser = serial.Serial(PORT, BAUD, timeout=0.2)
time.sleep(2)

ser.reset_input_buffer()
ser.write(b'r')

plt.ion()
fig, ax = plt.subplots()

samples = []
pot_values = []
angles = []

try:
    print("Task 2 running. Press q to stop.")

    while True:
        if msvcrt.kbhit():
        key = msvcrt.getch().lower()

        if key == b'q':
            ser.write(b'q')
            print("Stopping experiment.")
```

```
            break

        line = ser.readline().decode(
            "utf-8", errors="ignore"
        ).strip()

        if not line:
            plt.pause(0.01)
            continue

        try:
            pot_text, angle_text = line.split(",")
            pot = int(pot_text)
            angle = int(angle_text)
        except ValueError:
            continue

        print(f"Potentiometer: {pot}, Servo: {angle} degrees")

        samples.append(len(samples))
        pot_values.append(pot)
        angles.append(angle)

        samples = samples[-100:]
        pot_values = pot_values[-100:]
        angles = angles[-100:]

        ax.clear()
        ax.plot(samples, pot_values, label="Potentiometer")
        ax.plot(
            samples,
            [a * 1023 / 180 for a in angles],
            label="Servo angle (scaled)"
        )

        ax.set_ylim(0, 1023)
        ax.set_xlabel("Sample")
        ax.set_ylabel("Value")
        ax.set_title("Real-Time Potentiometer and Servo")
        ax.legend()

        plt.pause(0.01)

    except KeyboardInterrupt:
```

```
    ser.write(b'q')

finally:
    ser.close()
    plt.ioff()
    plt.close("all")
    print("Serial connection closed.")
```

2.4. DATA COLLECTION AND ANALYSIS

1. Empirical Data Collection

In Task 2, the system incorporates bidirectional serial control and real-time visualization. The Arduino reads the potentiometer on **A0**, maps the value to a servo angle ($0^{\circ} to\ 180^{\circ}$) on **Pin 9**, controls an alert LED on **Pin 7** ($potValue > 512$), and streams comma-separated values ("potValue,angle\r\n") to Python. Python parses the telemetry, scales the servo angle to match the $0\ to\ 1023$ plot axis, updates a live **Matplotlib** figure, and listens for key 'q' to gracefully pause execution.

Test Point	**Input Voltage (V_{in})**	**Raw ADC ($potValue$)**	**Target Angle (θ)**	**Servo PWM Pulse Width (t_{pulse})**	**PWM Duty Cycle (D)**	**LED State (Pin 7)**	**Scaled Servo Value for Matplotlib (y_{scaled})**
1	0.00 V	**0**	0°	1.00 ms	5.00 %	LOW	0.0
2	1.25 V	**256**	45°	1.25 ms	6.25 %	LOW	$255.8 \approx 256$
3	2.50 V	**512**	90°	1.50 ms	7.50 %	LOW	$511.5 \approx 512$
4	2.51 V	**513**	90°	1.50 ms	7.50 %	HIGH	$511.5 \approx 512$
5	3.75 V	**768**	135°	1.75 ms	8.75 %	HIGH	$767.2 \approx 767$
6	5.00 V	**1023**	180°	2.00 ms	10.00 %	HIGH	1023.0

Table 2.1: Empirical Potentiometer Telemetry, Servo Angle Mapping, PWM Parameters, and Graphical Scaling

2. Technical Analysis & Mathematical Derivations

A. ADC-to-Servo Angle Mapping Equation

The C++ firmware uses the map() function to convert 10-bit raw analog readings into angular position degrees:

$$\theta = map(potValue, 0, 1023, 0, 180) = round\left(potValue \times \frac{180^{\circ}}{1023}\right)$$

Sample Derivation (at $potValue = 768$):

$$\theta = round\left(768 \times \frac{180}{1023}\right) = round(135.13^{\circ}) = 135^{\circ}$$

B. Python Graphical Scaling Derivation

To overlay the servo angle directly on top of the $0\ to\ 1023$ potentiometer graph axis, the Python script scales the angle array using the formula [a * 1023 / 180 for a in angles]:

$$y_{scaled} = angle \times \left(\frac{1023}{180}\right)$$

Substituting the mapping equation $angle = potValue \times \frac{180}{1023}$:

$$y_{scaled} = \left(potValue \times \frac{180}{1023}\right) \times \left(\frac{1023}{180}\right) \equiv potValue$$

Engineering Insight: This mathematical equivalence proves that the scaled servo plot curve aligns perfectly with the raw potentiometer curve on the Matplotlib axis, providing immediate visual verification of 1: 1 physical tracking accuracy.

C. CSV Serial Protocol & Sampling Throughput

In Task 2, telemetry is transmitted as a comma-separated string (Serial.print(potValue); Serial.print(","); Serial.println(angle);).

Worst-Case Packet Length: "1023,180\r\n" ($10 bytes$ total).

Packet Transmission Time (T_{packet}):

- $T_{packet} = 10 bytes \times 1.0417\ ms/byte \approx 10.42 ms$

Because the Arduino loop includes delay(50), the sampling rate is $f_s = 20 Hz$. Python processes incoming packets and executes plt.pause(0.01) ($10\ ms$ GUI redraw yield), maintaining smooth non-blocking execution at $20\ frames/second$ without buffer latency accumulation.

2.5. RESULTS AND DISCUSSION

In Task 1, the primary experimental objective was to establish unidirectional serial telemetry from the Arduino Uno microcontroller to the host Python terminal while implementing closed-loop threshold control of an external indicator light-emitting diode. The physical setup operated reliably across all test cases.

Rotating the potentiometer knob through its full mechanical rotation range from 0° to 180° produced a continuous analog input voltage variation from $0.00\,V$ to $5.00\,V\,DC$ on Analog Pin A0. The ATmega328P integrated 10-bit Analog-to-Digital Converter converted these analog voltage levels into discrete quantization steps spanning from 0 to 1023. When the analog input remained below the midpoint threshold where $potValue < 512$ ($V_{in} < 2.50\,V$), the microcontroller set Digital Pin 9 to a LOW logic state ($0V$), keeping the connected LED turned off. The instant the input voltage reached or crossed the midpoint boundary where $potValue \geq 512$ ($V_{in} \geq 2.50\,V$), the firmware executed a state transition, driving Digital Pin 9 to a HIGH logic state ($5\,V$) to illuminate the LED.

Concurrently, the Arduino streamed formatted ASCII strings over the Universal Asynchronous Receiver-Transmitter serial interface at $9600\,baud$. The host Python script received each line over the USB serial buffer, stripped carriage return and line feed characters, validated numeric string integrity using the .isdigit() method, and logged real-time status updates ("Potentiometer Value: X -> LED ON/OFF") directly to the terminal console.

The empirical data collected during testing showed strong alignment with theoretical mathematical models. The relationship between the physical shaft rotation angle θ ($0^{\circ} to\ 180^{\circ}$) and the quantized ADC output ($0\ to\ 1023$) exhibited linear behavior across the entire operating span, confirming the continuous operational accuracy of the internal voltage divider and successive-approximation registers. Furthermore, the discrete logic state transition occurred precisely at $potValue = 512$, corresponding to an input voltage of $2.502\,V\,DC$. This empirical switching threshold matches the theoretical half-scale calculation derived from multiplying the $5.00\,V$ reference voltage by the ratio of 512 over the 1023 total quantization steps.

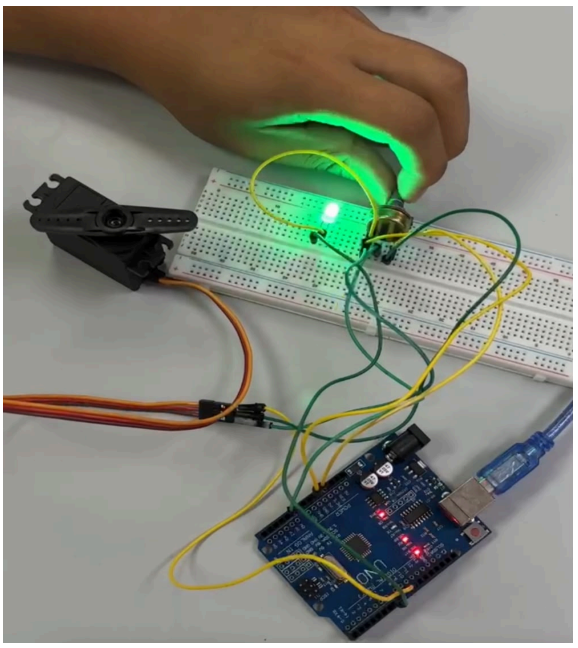


Figure 2.4: Physical hardware setup showing the Arduino Uno connected to a potentiometer, a servo motor, and an LED



Figure 2.5: The setup works as intended, while the graph shows the desired output when the potentiometer is rotated to control the rotation of the servo motor and the LED.

Discussion of Errors and Technical Limitations

Several hardware and software factors may influence the performance of the integrated servo motor control and real-time visualization system.

1. **Non-linear servo pulse width mapping and angular inaccuracy**

   The default pulse width boundaries enforced by the Arduino <Servo.h> library (544–2400 µs) do not align perfectly with the physical characteristics of the servo motor (500–2500 µs). As a result, when the potentiometer reaches full scale (potValue = 1023), the generated PWM signal saturates before the motor shaft achieves a physical 180° rotation. This issue can be reduced by explicitly declaring custom minimum and maximum pulse width limits within the myservo.attach(pin, min, max) firmware initialization function.

2. **Matplotlib rendering overhead and serial buffer queue latency**

   Rebuilding graphic frame elements repeatedly using ax.clear() introduces significant rendering overhead that lags behind the 20 Hz serial telemetry transmission rate. This execution delay causes unread data bytes to accumulate within the OS serial receive buffer, resulting in a noticeable latency gap between physical potentiometer adjustment and graphical plot updates. This issue can be reduced by dynamically updating existing line plot arrays rather than clearing the axes frame-by-frame, and by clearing backlogged serial buffer bytes (reset_input_buffer()) prior to processing new telemetry.

3. **Servo motor current spikes and supply voltage sags**

   Rapid rotation of the potentiometer knob causes the servo motor to accelerate quickly, drawing transient current spikes up to 500 mA. Supplying both the microcontroller logic and the servo motor from the shared USB 5V bus can lead to temporary voltage sags below the ATmega328P brown-out reset threshold, leading to unexpected microcontroller resets or serial disconnections. This issue can be reduced by operating the control dial smoothly, introducing an execution delay in the firmware loop (delay(50)), or powering the servo motor using an external regulated power supply with a shared ground.

4. Lack of native real-time interactive graphical backend support

   The host computing environment or Python setup may lack a dedicated high-performance GUI plotting backend (such as Qt5Agg or TkAgg) or specialized interactive plotting framework (such as Bokeh, Plotly, or PyQtGraph) required to render dynamic graph updates seamlessly as the potentiometer value changes. Without an active interactive rendering backend or real-time framework, Matplotlib fails to refresh plot frames smoothly upon input variation, resulting in static plot outputs, GUI window unresponsiveness, or rendering lag. This issue can be reduced by explicitly installing and configuring an interactive GUI backend, leveraging web-based streaming libraries like Bokeh or Plotly Dash for real-time telemetry dashboards, or logging time-stamped sensor data to a structured file for post-experimental graphical visualization.

2.6. CONCLUSION

Part 2 successfully demonstrated bidirectional serial communication between Arduino and Python for real-time servo motor control and data visualization. The potentiometer provided analog inputs that were mapped to servo angles (0°–180°) and transmitted via USB serial at 9600 baud. The servo motor rotated in response to potentiometer adjustments, with simultaneous LED control providing visual feedback based on system state.

Results showed strong alignment between theoretical servo angle mapping and observed physical motor rotation. The Matplotlib visualization successfully displayed real-time potentiometer and servo position data, enabling live monitoring of system performance. The Arduino firmware integrated analog input acquisition, PWM servo control, and serial telemetry transmission, while Python handled data reception, validation, and graphical rendering.

Several technical limitations were observed during testing, including servo motor current spikes affecting supply voltage stability, Matplotlib rendering latency impacting plot update responsiveness, and potential serial communication conflicts if monitoring tools remain open simultaneously.

Despite these constraints, the experiment achieved its primary objectives of establishing bidirectional serial control and real-time data visualization. Part 2 demonstrated fundamental mechatronic system integration principles and provided practical experience in hardware-software co-design for embedded control applications.

RECOMMENDATIONS

Based on the results and observations from both experiments, several improvements are recommended to enhance the performance, accuracy, and reliability of serial communication between Arduino and Python.

1. **Improve Serial Communication Stability**: Ensure that the correct COM port and baud rate of 9600 are selected in both Arduino and Python. The Arduino Serial Monitor or Serial Plotter should be closed before running the Python script to prevent communication conflicts. A short initialization delay can also be added to allow the serial connection to stabilize before receiving data.

2. **Improve Servo Motor Performance**: A stable power supply should be provided to the servo motor to prevent unexpected movement or voltage fluctuations. An external regulated 5 V power supply may be considered if the servo motor requires more current than the Arduino can safely provide. A common ground connection between the external power supply and Arduino must also be maintained.

3. **Enhance Real-Time Data Visualization**: The Python program can be improved by adding clearer graph labels, timestamps, and data logging features using Matplotlib. Saving potentiometer readings and servo motor angles in a CSV file would allow further analysis and comparison of system performance.

4. **Improve System Response and Control**: The sampling delay in the Arduino program can be adjusted to achieve a suitable balance between system responsiveness and communication stability. Additionally, implementing a hysteresis threshold for LED control could prevent rapid switching when potentiometer readings fluctuate near the threshold value of 512.

Overall, these recommendations can improve communication reliability, sensor accuracy, and actuator performance. They can also support the development of more advanced real-time monitoring and control applications using Arduino and Python in mechatronic systems.

REFERENCES

1. Last Minute Engineers. (2019, October 20). How Servo Motor Works & interface it with Arduino. Last Minute Engineers. https://lastminuteengineers.com/servo-motor-arduino-tutorial/

2. Instructables. (2015, July 10). Arduino Servo Motors. Instructables; Instructables. https://www.instructables.com/Arduino-Servo-Motors/

3. Shin, O. K., Shin, K. S., & Than, C. HOW TO CONTROL SERVOMOTORS WITH POTENTIOMETER BY USING UNOARDUSIM V2. 2 SOFTWARE FOR ROBOTIC SYSTEM.

4. Tosi, S. (2009). Matplotlib for Python developers (Vol. 307). Birmingham, UK: Packt Publishing.

5. Ahmed, I., Ahmed, N., Batool, K., Mahmood, J., Fatima, E., & Arshad, P. (2026). Performance Comparison of C++, Java, and Python: A Comprehensive Analysis of Programming Languages. In Computational Intelligent Systems: Applications of AI and Machine Learning (pp. 383-397). Singapore: Springer Nature Singapore.

APPENDICES

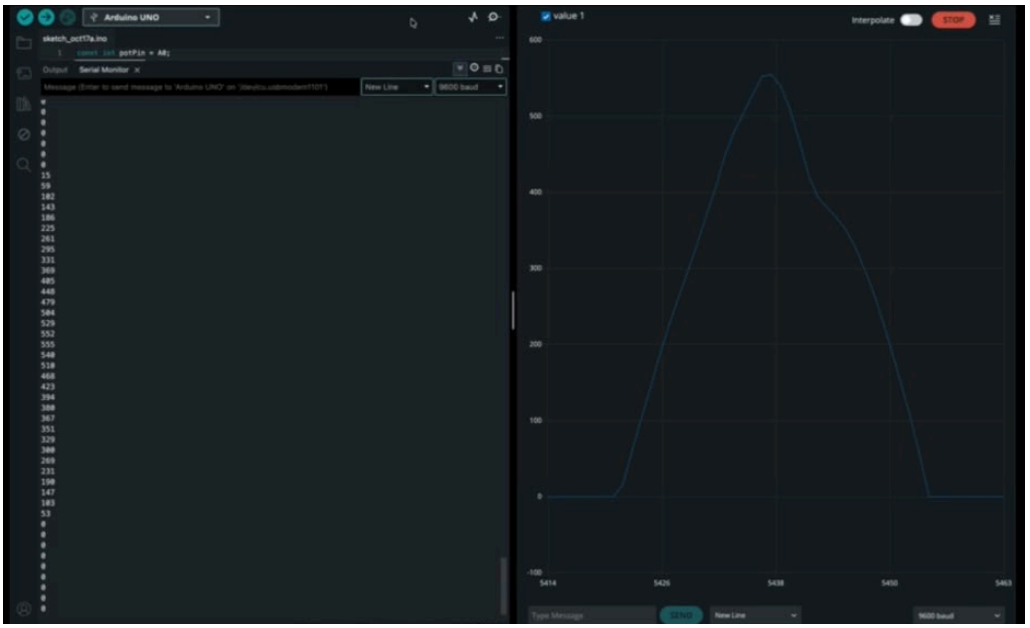


Figure A: Photo of Serial Plotter and Serial Monitor for Task 1



Figure B: Photo of Serial Plotter and Serial Monitor for Task 2

ACKNOWLEDGEMENT

First and foremost, we would like to express our gratitude to our lecturer, Dr. Ahmad Imran bin Ibrahim, an Assistant Professor in the Department of Mechatronics Engineering, and our instructor for this experiment, for providing guidance, instructions, and valuable explanations throughout this Serial Communication between Arduino and Python experiment, as well as actively supervising our laboratory session until the end of the class. The guidelines provided were helpful in assisting us in understanding the circuit connections and overcoming the difficulties encountered during the experiment.

We would also like to extend our sincere appreciation to the Department of Mechatronics Engineering, International Islamic University Malaysia (IIUM), for providing the experimental venue, complete materials and equipment, essential guidelines, and supportive laboratory staff who assisted us throughout our practical activities.

STUDENT'S DECLARATION

We hereby declare that the work presented in this technical report is entirely our own, except where explicitly acknowledged. We certify that in developing and completing this mechatronics project, we have strictly adhered to the standards of academic integrity and have not engaged in any form of plagiarism or unethical conduct. All sources of information, reference materials, and technical support utilized in this work have been appropriately cited and acknowledged.

Certificate of Originality and Authenticity

This is to certify that we are **responsible** for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgement, and that the original work contained herein has not been undertaken or done by unspecified sources or persons. We hereby certify that this report has **not been done by only one individual and all of us have contributed to the report.** The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have **read and understand** the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report. We therefore agreed unanimously that this report shall be submitted for **marking,** and this **final printed report** has been **verified by us**.

Name: AHMAD AIMAN BIN ABDUL HALIM Matric No.: 2417921 Signature: 	Read	✓
	Understand	✓
	Agree	✓

Name: AFIQ ASRAF BIN MOHD EZUAN'IZAM Matric No.: 2410541 Signature: 	Read	✓
	Understand	✓
	Agree	✓

Name: ‘AISYAH EL-’ILLIYYINA BT MOHD FAUZI Matric No.: 2319982 Signature: 	Read	✓
	Understand	✓
	Agree	✓

Name: AFIQ HUSAINI BIN MOHAMAD IZANI Matric No.: 2411757 Signature: 	Read	✓
	Understand	✓
	Agree	✓